กติกาสอบ & คะแนน

เวลา	1 ชม. · 09:00–10:00 · 14 ต.ค.
รูปแบบ	MCQ ทั้งหมด (ตัวเลือก A–E)
พกได้	**A4 หน้า–หลัง 1 แผ่น เท่านั้น**
ขอบเขต	Week 1–3 (Lab 1–3) · ไม่รวม Lab 4
หลังสอบ	10:00–12:00 Lab 4 Image Captioning

คะแนน: Lab 1–4 ละ 4% = 16% (กลุ่ม 3 / เดี่ยว 1) · Quiz 10% → รวม 25% + bonus 1% (เพดาน 25%) · ข้อสอบเก่า 8 ต.ค. 2025 = **32 ข้อ ข้อละ 1 คะแนน**

IoT คืออะไร · 9 Characteristics

- **IoT / IoE** = เครือข่ายของ things + platform ที่คุยกันเองผ่าน wireless โดยไม่ต้องมีมนุษย์สั่ง (without human intervention) → efficiency, performance, safety
- **3 ประโยชน์:** Communication · Control & Automation · Cost Saving
- **9 คุณลักษณะ:** Connectivity · Intelligence · Dynamic Nature · Enormous Scale · Sensing · Heterogeneity · Security · Unique Identity · Integration into Information Network

กับดัก: ถามว่าอะไร **ไม่ใช่** คุณลักษณะ IoT → ตอบ **High latency** · **Scalability** คือคำตอบลวง เพราะ = **Enormous Scale**

ภาพรวม 4 ชั้นของระบบ IoT

1. **Devices & Sensors** — เก็บข้อมูลจริง · MCU/edge: Arduino, ESP32, Raspberry Pi
2. **Comm & Networking** — Wi-Fi, BLE, ZigBee, LoRa, 5G · IPv6, 6LoWPAN, MQTT, CoAP
3. **Data Processing & Platforms** — edge/cloud · storage, filtering, analytics, AI/ML → raw data → information
4. **Applications** — Smart Home · Smart City · Healthcare · Transportation

Edge device: Raspberry Pi, ESP32, Arduino, Jetson Nano — Desktop PC ไม่นับ

Use Cases — Smart City

	SINGAPORE	BARCELONA
จุดเน้น	citizen-centric ทั่วประเทศ	urban infrastructure & sustainability
โครงการ	Smart Nation	Smart City
แพลตฟอร์ม	รัฐบาลผลักดัน	Sentilo / CityOS (open data)
ตัวอย่าง	MRT/bus tracking · e-payment · telemedicine+wearables · automated immigration (ใบหน้า+ม่านตา) · smart water meter · environmental modelling · robotics/drones	smart parking · smart lighting · smart waste · air quality · smart irrigation

Communication Models ออกสอบแน่

MODEL	ลักษณะ · โปรโตคอล
D2D Device-to-Device	คุยกันตรง ๆ ไม่ผ่าน cloud/gateway · **MQTT · CoAP · WebSocket · BLE** · smartwatch ส่ง step → มือถือ (BT)
D2C Device-to-Cloud	ยิงขึ้น cloud ตรง ๆ ไม่ผ่าน gateway · **REST (HTTP/S) · MQTT · gRPC** · fitness tracker → cloud · thermostat → AWS IoT Core
D2G Device-to-Gateway	เซนเซอร์ → gateway (Pi/router/มือถือ) · **MQTT · Zigbee · BLE** · smart home/farm → Pi
Data sharing (C2C)	Cloud-to-Cloud · Fitbit Cloud → Google Fit → sw. · Azure IoT Hub → SAP

ตัวตัดสิน: มี gateway? → ไม่มี + ขึ้น cloud = D2C · ไม่มี + ไม่ขึ้น cloud = D2D · มี = D2G · cloud↔cloud = C2C

Data Storage & Database

1 · **ที่เก็บ:** On-Device (SD/embedded DB) · Edge (gateway, latency ต่ำ) · Cloud (scale, big data & ML)
2 · **DATA MODEL:** SQL — MySQL, PostgreSQL · NoSQL: Key-Value Redis, DynamoDB · Document MongoDB, CouchDB · **Time-Series** InfluxDB, TimescaleDB · Graph Neo4j · Object Storage (รูป/วิดีโอ) · Vector DB
3 · **USE CASE:** Streaming — Kafka, RabbitMQ, MQTT broker · Analytical/Data Lake — BigQuery, Redshift, Snowflake · IoT-specific — AWS Timestream, Azure Cosmos DB, Firebase

เวลาต่อเนื่อง → **Time-Series** · โครงสร้างตายตัว → **SQL** · รูป/วิดีโอ → **Object Storage**

Processing Models

MODEL	ข้อดี	ข้อเสีย	ใช้ที่ไหน
Cloud-centric	พลังสูง · big data · อัปเดตโมเดลง่าย · เทรน ML	latency สูง · ต้องมีเน็ต · เสี่ยง privacy	Smart City/Agri, healthcare analytics
Edge-only	latency ต่ำสุด · offline ได้ · privacy ดี	ทรัพยากรจำกัด · โมเดลเล็ก · อัปเดตยาก	รถไร้คนขับ, wearable, CCTV real-time
Hybrid	สมดุล latency · ลด bandwidth	ซับซ้อน · แพง · secure 2 ฝั่ง	Smart Factory, predictive maintenance

สูตรเลือกข้อ: มีทั้ง real-time ปลายทาง + วิเคราะห์ ML ก้อนใหญ่ → **Hybrid** ทุกครั้ง

Spatial Filtering vs Convolution

- **Filtering (correlation)** = ไถ kernel หาผลรวมผลคูณ ตรง ๆ
- **Convolution** = **พลิก kernel 180° ก่อน** → output คล้าย kernel · สมมาตร = เท่ากัน · ใช้ `cv2.filter2D`
- **Box/Average (cv2.blur)** = เบลอ · kernel ใหญ่ขึ้น = **เบลอแรงขึ้น + ช้าลง** (ไม่ใช่คมขึ้น/fps ขึ้น)
- **Gaussian** = เบลอนุ่ม ถ่วงกลาง · **Laplacian** = เหลือแต่ขอบ (อนุพันธ์อันดับ 2) ขยาย noise
- **Sobel/Prewitt** = ขอบตามแกน x/y · **Sharpen** = คมขึ้น

EDGE VS FOG VS CLOUD

	EDGE	FOG	CLOUD
ประมวลผล	อุปกรณ์/gateway	local server	data center
Latency	ต่ำสุด real-time	ต่ำ–กลาง	สูงกว่า
เชื่อมต่อ	offline ได้	ต้องมี LAN	ต้องมี internet

IoT Levels 4 ระดับ สับสนบ่อย

ระดับ	ทำอะไร	ตัวอย่าง
1 Measurement	เก็บ raw data ด้วยเซนเซอร์	เซนเซอร์ดินตรวจว่าดินแห้ง
2 Automation	ทำเองจากค่าที่วัด (if-else/threshold)	ปั๊มน้ำเปิดเอง · ประตูเปิดเอง
3 Innovation	บริการ/โมเดลธุรกิจใหม่ + AI/analytics	ผูกทะเบียน+จ่ายอัตโนมัติ+ปรับราคาตามเวลา
4 Cloud Storage	เก็บ แชร์ วิเคราะห์ ส่วนกลาง	เก็บข้อมูลวิเคราะห์ผลผลิต

Automation vs Innovation: อยู่ในลิสต์→ไม้กั้นเปิดเอง = **Automation** · เพิ่มจ่ายผ่านแอป+ปรับราคาตามเวลา = **Innovation**

User-facing IoT Apps — 3 หน้าที่

1 **Visualization** — ค่าสด real-time · dashboard parking อัปเดตทันที · เกมโชว์ท่า "ROCK" ทันที
2 **Notification & alerts** — push เอง (threshold/anomaly) · HR > 120 bpm · เกมขึ้น "You Win!" = Notification
3 **Control loop** — สั่งแล้วสถานะเปลี่ยนจริง · "Turn on AC" · เกม: ปุ่มเริ่มรอบใหม่

Performance: latency > 1 วินาที = เกมเล่นไม่ได้ · 2 กล้อง (แบบ LINE) ต้อง sync + ส่ง state → D2C/C2C

โปรโตคอลในวิชานี้

PROTO	รูปแบบ	ใช้เมื่อ
MQTT	Pub/Sub ผ่าน broker · TCP	เซนเซอร์มาก แบนด์วิดท์ต่ำ · D2D/G/C
CoAP	REST-like บน UDP	อุปกรณ์จิ๋ว พลังงานต่ำ · D2D
HTTP/S	REST Req/Res	cloud/API · MJPEG ผ่าน Flask (แล็บ 1&3)
WebSocket	สองทาง คาไว้	dashboard real-time
gRPC	RPC บน HTTP/2 · binary	service คุยกันในระบบใหญ่
ZigBee/BLE/LoRa	วิทยุระยะสั้น–ไกล ไฟต่ำ	D2D/D2G smart home, farm

ชั้น (layer)	โปรโตคอล
Application	HTTP · MQTT · AMQP · XMPP · DDS · CoAP
Transport	TCP · UDP
Network	IPv6 · RPL · 6LoWPAN
Data link / Physical	802.15.4 · BLE · NFC/RFID · Wi-Fi

6LoWPAN = IPv6 บน 802.15.4 · **ไร้สาย:** Wi-Fi เร็ว/สั้น–กลาง/กินไฟ · BLE ใกล้/ไฟต่ำ · ZigBee mesh · LoRa ไกล km/ไฟต่ำ/data น้อย · 5G latency ต่ำ

คู่ที่มักถูกสลับ

เทียบ	ต่างกันตรงไหน
Automation / Innovation	ทำเองตามกฎจากค่าที่วัด / สร้างบริการ+โมเดลธุรกิจใหม่
D2C / D2G	ต่อ cloud ตรง ๆ / ผ่าน gateway (ต่อเน็ตเองไม่ได้)
D2D / Data sharing	อุปกรณ์↔อุปกรณ์ / cloud↔cloud (Fitbit→Google Fit)
Edge / Fog / Cloud	ใกล้สุด→กลาง→ไกลสุด ; latency ต่ำ→สูง ; offline ได้แค่ edge
Cloud-centric / Edge-only / Hybrid	ส่งดิบขึ้น cloud / ทำบนเครื่อง / เบื้องต้นบนเครื่อง+หนักบน cloud
SQL / NoSQL	ตาราง schema ตายตัว / KV, Document, Time-Series, Graph, Vector ยืดหยุ่น
MQTT / HTTP / CoAP	Pub/Sub+broker (TCP) / Req-Res (TCP) / ≈HTTP เบา (UDP)
TCP / UDP	รับประกัน+เรียง ช้ากว่า / ไม่รับประกัน เร็ว เบา
MCU / SBC	ESP32 เฟิร์มแวร์เดียว real-time ไฟต่ำ / RPi Linux หลายโปรแกรม
Precision / Recall	ในที่ทายว่าใช่ ถูกกี่ส่วน / ในที่ใช่จริง จับได้กี่ส่วน (F1 = harmonic)
Visualization / Notif. / Control	ดูสด / push เตือน / สั่งอุปกรณ์
Filtering / Convolution	ไม่พลิก kernel / พลิก kernel 180° (สมมาตร = เท่ากัน)
greedy / beam	argmax ทีละตำแหน่ง เร็ว / เก็บ k เส้นทาง แม่นกว่า ~k เท่า
CAPEX / OPEX	เงินลงทุนแรก / ค่าดำเนินการต่อเนื่อง
FAR / FRR	False Accept (ผิดแต่ PASS=เสี่ยง) / False Reject (ถูกแต่ FAIL)

รายการ "ข้อใดไม่ใช่" — ไล่ให้ครบ

หัวข้อ	ครบถ้วน
ประโยชน์ IoT (3)	Communication · Control&Automation · Cost Saving
ลักษณะ IoT (9)	Connectivity · Intelligence · Dynamic Nature · Enormous Scale · Sensing · Heterogeneity · Security · Unique Identity · Integration into Info Network
ชั้น IoT (4)	Devices&Sensors · Comm&Networking · Data Processing&Platforms · Applications
Comm model (4)	D2D · D2C · D2G · Data sharing (C2C)
Processing (3)	Cloud-centric · Edge-only · Hybrid (+ Edge/Fog/Cloud)
Storage (3 มุม)	ที่เก็บ · Data model · Use case
OCR (4 กลุ่ม)	Basic · Cloud · Advanced · Native/Mobile
IoT Levels (4)	Measurement · Automation · Innovation · Cloud Storage
User-facing (3)	Visualization · Notification&Alerts · Control loop
Evaluation (7)	Accuracy · Automation&Responsiveness · Innovation&Usefulness · Cloud Integration · System Performance · UX&Presentation · Cost
AI-specific (6)	Model Accuracy · Inference Latency · Resource Efficiency · Robustness&Fairness · Explainability · Maintainability
OCR 3 ชั้น	Detection · Recognition · Decoding
status log (3)	ABSENT(0) · NORMAL(1) · SUSPICIOUS(≥2)

คำสั่ง & ไลบรารีที่ใช้ในแล็บ

- **venv:** `python3 -m venv --system-site-packages env-labN` (เห็นแพ็กเกจระบบ เช่น picamera2) → `source env-labN/bin/activate` → `pip3 install -r requirements.txt` → `deactivate`
- **รัน:** `python3 lab1_guide.py | lab2-app.py | lab3-app.py` → เปิด `http://<ip>:5000`
- **DB:** `sqlite3 plates.db "SELECT * FROM events;"` · เริ่มใหม่ `rm plates.db`
- **กล้อง:** `cat /etc/os-release` · `rpicam-hello` / `rpicam-still -o test.jpg` / `rpicam-vid -o test.h264` · เปิดกล้องรุ่นเก่า `raspi-config → Interfacing → Camera`
- **ฟอนต์ไทย:** `sudo apt install fonts-noto fonts-noto-cjk`
- **ไลบรารี:** Lab1 = **OpenCV + Picamera2** · Lab2 = **EasyOCR + SQLite** · Lab3 = **MediaPipe + Flask/OpenCV**
- **Flask:** `app.run(host="0.0.0.0", port=5000)` · `Response(gen(), mimetype="multipart/x-mixed-replace; boundary=frame")`

คำศัพท์สำคัญ

PDPA กฎหมายข้อมูลส่วนบุคคล (เหตุผลที่ต้องเบลอหน้า/ประมวลผลในเครื่อง) · **Pub/Sub** ผู้ส่ง-รับไม่รู้จักกัน คุยผ่าน broker ตาม topic · **MQTT** port **1883** · **Gateway** ตัวกลาง · **Edge/Fog/Cloud** ใกล้→ไกล · **Time-Series DB** · **CAPEX/OPEX** · **Latency** เวลาตอบสนอง · **Throughput** ปริมาณต่อเวลา · **Uptime** เวลาที่ระบบทำงาน · **XAI** อธิบายผลโมเดล · **MLOps** วงจรชีวิตโมเดล · **Heterogeneity** อุปกรณ์หลากหลาย · **free tier** โควตาฟรี · **false positive/negative** ทายเกิน/ทายพลาด

อาการ → สาเหตุ → วิธีแก้

อาการ	สาเหตุ / แก้
fps 0–15	โหลด CascadeClassifier ในลูป → ย้ายไประดับโมดูล + ย่อภาพ + ตรวจทุก N เฟรม
ภาพกระตุก ทั้งที่ fps เฉลี่ยสูง	jitter (เฟรมตรวจช้า/ใช้ cache เร็วสลับ) → **frame pacing**
log timestamp ผิดรูปแบบ	`isoformat()` มีไมโครวินาที → `timespec="seconds"`
ข้อมูลไม่เข้า DB	ลืม `con.commit()`
SQL ผิดพลาด/ถูกโจมตี	ต่อสตริง → ใช้ `?` parameter
ไทยไม่ขึ้นบนภาพ OCR	ไม่มีฟอนต์ → `apt install fonts-noto`
กรอบ OCR เพี้ยน (ภาพใหญ่)	OCR บนภาพที่ thumbnail แต่วาดบนต้นฉบับ → คูณสัดส่วนกลับ
ท่ามือผิดเมื่อมือเอียง	วิธี y ใช้ได้เฉพาะมือตั้งตรง → ใช้ระยะจากข้อมือ
กด Battle แต่ใช้ท่าเก่า	ไม่ล้าง `character` → `else: character=""`
เปิด /battle ตรง ๆ ขึ้น 405	route รับเฉพาะ POST
สีเพี้ยนใน MediaPipe	ลืม BGR→RGB

IoT Evaluation 7 หัวข้อ ออกหลายข้อ

หัวข้อ	ความหมาย · ตัวอย่าง
1 Measurement Accuracy	วัดถูกไหม · soil ±2% · detect ≥ 85% · เทียบ ground-truth ที่ calibrate ตาม datasheet
2 Automation & Responsiveness	ทำเองจากค่าที่วัด + ตอบไว · ไฟติด 0.5 วิ · เตือน HR > 120 ถูก 90% · alert 160 bpm
3 Innovation & Usefulness	คุณค่าใหม่เกินวัด/ควบคุม · ถังขยะ AI · ห้องเรียนวัด attention
4 Cloud Integration & Data Handling	ขึ้น cloud เก็บ/แสดง/วิเคราะห์ · PM2.5 → Firebase → แอป · Node-RED+MQTT · connectivity ≥ 95% · เกณฑ์ 5 = ≥95%/ไม่สูญหาย · 0 = ไม่มี/ไม่เก็บ
5 System Performance	efficiency+reliability+scalability · uptime 95%/7 วัน · 4 กล้อง latency < 1 วิ · แบต 12 ชม.
6 UX & Presentation	ใช้ง่าย ชัด · Chart.js/Node-RED · ปุ่ม ON/OFF · live demo · **เมตริกที่ถูก = เวลาเรียนรู้การใช้แอป** (ไม่ใช่ CPU/API/โควตา)
7 Cost Efficiency & Feasibility	HW · free-tier vs paid · CAPEX/OPEX · cost-benefit · sustainability · farming < 2,000฿ · parking ~8,000฿ + AWS ต่อ request

AI-Specific Evaluation 6 หัวข้อ

1. **Model Accuracy** — Precision / Recall / F1
2. **Inference Latency** — edge vs cloud
3. **Resource Efficiency** — Memory, CPU/GPU → ข้อ **"แบต 12 ชม."** ตอบข้อนี้
4. **Robustness & Fairness** — ทนทุกสภาพ ไม่มีอคติ
5. **Explainability (XAI)** — อธิบายเหตุผลได้ → **หมอ/ผู้ใช้เชื่อถือผล AI**
6. **Maintainability** — retrain/MLOps → **โมเดล 2023 เจอโรคพืชใหม่ 2025 แล้วแย่ = ข้อนี้**

Lab 1 · Face Detection + Blur

ไปป์ไลน์: Frame capture (OpenCV) → Haar Cascade → Privacy Blur → Logging → Flask MJPEG Stream

- สายกล้อง Pi = **MIPI CSI-2 serial** (ไม่ใช่ USB/UVC, I²C, SPI, HDMI)
- `create_preview_configuration(main={"format":"XRGB8888","size":(640,480)})` → **XRGB8888 = 4 ไบต์/พิกเซล** (R,G,B+X; หน่วยความจำเรียง B,G,R) · `capture_array()`
- สตรีมขึ้นเว็บ = **Flask + HTTP (MJPEG)** ไม่ใช่ MQTT/WebSocket/gRPC
- **Haar เร็วเพราะ** Integral Image (sum เร็ว) + Cascade Stages (ตัดพื้นหลังไว) · Sliding Window ไถทั่วภาพ
- **scaleFactor ↑** เร็วแต่พลาดขนาด · **minNeighbors ↑** FP น้อย · **minSize ↑** เร็ว
- **fps ต่ำแก่ที่:** โหลด Cascade **ครั้งเดียวนอกลูป** (ต้นเหตุหลัก) · ย่อภาพ · ตรวจทุก N เฟรม + cache กรอบ · **frame pacing** แก้ jitter
- log ISO 8601 ต้อง `timespec="seconds"` (ไม่ใส่ = มีไมโครวินาที) · `open("log.txt","a")`
- status: **ABSENT=0 · NORMAL=1 · SUSPICIOUS≥2** · เขียน log ทุก 1 วิ / เมื่อ status เปลี่ยน (ไม่ทุกเฟรม)
- เช็ค OS `cat /etc/os-release` · Bullseye `libcamera-*` → Bookworm `rpicam-*`

```
f = cv2.CascadeClassifier(cv2.data.haarcascades
    + 'haarcascade_frontalface_default.xml')  # load once
faces = f.detectMultiScale(gray, scaleFactor=1.1,
  minNeighbors=5, minSize=(30,30))
for (x,y,w,h) in faces:
  frame[y:y+h,x:x+w] = cv2.blur(frame[y:y+h,x:x+w],(25,25))
```

fps = 1 / เวลาต่อเฟรม (s) → 50 ms = 20 fps · ในแล็บช่วงหน้าเริ่มเบลอ = **11–25 fps**

Lab 2 · OCR อ่านป้ายทะเบียน

raw image → ข้อมูลใช้ได้ · ไลบรารี = **EasyOCR** (ไม่ใช่ OpenCV/NumPy/Matplotlib/Pandas) · โชว์บน Flask/Gradio · ใช้กับ smart logistics, parking, document digitization

- **OCR 3 ขั้น:** **Detection** (CRAFT หา bbox 4 มุม) → **Recognition** (CRNN) → **Decoding**
- **4 กลุ่ม:** Basic (Tesseract 5–20MB, EasyOCR 30–80MB ฟรี เบา แต่ไทยแย่ ไม่มี layout) · Cloud (Google Vision/Azure/AWS Textract แม่น แต่ต้องเน็ต+จ่าย+privacy) · Advanced (PaddleOCR, MMOCR, DocTR, Keras-OCR หนัก ต้อง GPU) · Native/Mobile (iOS Vision, ML Kit offline ผูกแพลตฟอร์ม)

```
Reader(['th','en'], gpu=False, quantize=True)
readtext(img, allowlist=ALLOWLIST,
  text_threshold=0.7, low_text=0.4)
# [(bbox, text, confidence), ...]
```

- **text_threshold ↑** = ขยะมั่วทั่วภาพลด · **low_text ↓** = จับตัวจางได้ (ขยะเพิ่ม) · **link_threshold** เชื่อมตัวอักษรเป็นกล่อง
- **allowlist** (ไทย+0–9) ลดอ่านผิดประเภท แต่ไม่ช่วยความเร็ว/contrast/การหมุน
- **greedy** (เร็ว ย้อนแก้ไม่ได้) vs **beamsearch** (k เส้นทาง แม่นกว่า ~k เท่า, beamWidth 5)
- **เร่ง:** ย่อภาพ (งาน conv ตามพิกเซล) · greedy · `rotation_info=[0]` (ลดงาน ~3 เท่า) · quantize · โหลด Reader ครั้งเดียวนอกลูป · วัด ms ก่อน/หลัง
- preprocess: `ImageEnhance.Contrast(img).enhance(1.25)` · `UnsharpMask(radius=1.0,percent=120,threshold=3)` · `thumbnail((800,800))` คงสัดส่วน ไม่ขยาย
- **SQLite:** ไฟล์เดียว · `CREATE TABLE IF NOT EXISTS` · `INSERT OR IGNORE` · **`?` กัน SQL injection** · ลืม `con.commit()` = **ไม่บันทึก** · บันทึกทุกครั้ง (PASS+FAIL)
- ประกอบทะเบียน: กรอง conf → min_y,min_x จาก 4 มุม → sort y → แถวบน y ห่าง < **40px** → sort x → join ไม่เว้นวรรค → เทียบ DB ตรงเป๊ะ
- Base64: `b64encode(buf).decode('ascii')` → `<img src="...">`

เมตริก Accuracy = exact-match ทั้งป้าย (ผิดตัวเดียว = FAIL) · **FAR/FRR** = False Accept/Reject · **CER** = (S+D+I)/N (ไม่ใช่ ถูก 85% ไม่ใช่ fps) · **WER** = ระดับคำ · **Precision** TP/(TP+FP) · **Recall** TP/(TP+FN) · **F1** 2PR/(P+R) · **BLEU/OKS** ไม่ใช่เมตริก OCR

Lab 3 · MediaPipe + เป่ายิ้งฉุบ

ไปป์ไลน์: Picamera2 init → capture + BGR→RGB → Hand Landmark → y ของ TIP/PIP → classifier() → วาด → encode MJPEG → Flask → HTML

- **21 landmarks/มือ** (ไม่ใช่ 19/20/22/23) · index 0 = WRIST · lm.x,lm.y เป็น 0–1 → คูณ W,H
- ข้อนิ้ว โคน→ปลาย: **MCP → PIP → DIP → TIP**
- **index:** 0 WRIST · โป้ง 1–4 (4=TIP) · ชี้ 5–8 · กลาง 9–12 · นาง 13–16 · ก้อย 17–20 → **TIP = 4,8,12,16,20 · PIP = 6,10,14,18 · MCP = 5,9,13,17**
- **y = 0 ขอบบน เพิ่มลงล่าง** → มือตั้งตรง: **นิ้วเหยียด = TIP.y < PIP.y**
- **ทนมือเอียง:** ใช้ระยะจากข้อมือ `hypot(dx*W, dy*H)` เหยียดเมื่อ `dist(tip,wrist) > dist(pip,wrist)`
- game: `beats={'ROCK':'SCISSORS','SCISSORS':'PAPER','PAPER':'ROCK'}` · เท่ากัน=Tie · LOVE ไม่นับแพ้ชนะ
- **จับเวลา: 4a** = detect อย่างเดียว · **4b** = detect+feature+classifier · **4b−4a = เวลา classifier** · เรียกครั้งแรกช้า (warm-up) → ใช้ค่าเฉลี่ย
- Flask: `/battle` POST เท่านั้น (GET = 405) · `global character` เมื่อ assign · ไม่มีมือ → `character=""` · num_hands=2 → มือสุดท้ายชนะ

```
HandLandmarkerOptions(model_asset_path="hand_landmarker.task",
  num_hands=2, running_mode=IMAGE)  # detect(mp.Image(SRGB, rgb))
up = lambda t,p: t < p                  # นิ้วเหยียด
if not any(f): return "ROCK"
if all(f): return "PAPER"
if f[0] and f[1] and not f[2] and not f[3]: return "SCISSORS"
```

กับดักที่พลาดบ่อย

01 **High latency** ไม่ใช่คุณลักษณะ IoT — Scalability คือ (Enormous Scale)
02 Desktop PC ไม่ใช่ edge device · Jetson Nano เป็น
03 กล้อง Pi = MIPI CSI-2 · สตรีม = Flask + HTTP
04 kernel ใหญ่ขึ้น = เบลอแรงขึ้น · Laplacian = เหลือแต่ขอบ
05 Convolution พลิก kernel · filtering ไม่พลิก
06 CCTV ที่ถูก: buffer สั้น → edge จำหน้า+กรอง event → cloud analytics+เก็บยาว
07 D2C = ไม่ผ่าน gateway — จับคำ **"directly"**
08 ข้อดี Cloud-centric = พลัง+เทรน ML ไม่ใช่ latency ต่ำ/privacy
09 ข้อเสีย Cloud OCR = network dependency + privacy
10 ขยะ OCR ทั่วภาพ → เพิ่ม text_threshold
11 Innovation = บริการ/มูลค่าใหม่ ไม่ใช่ if-else
12 **"You Win!"** ทันที = Notification ไม่ใช่ Visualization
13 MediaPipe = 21 จุด · นิ้วเหยียด TIP.y < PIP.y
14 UX metric = เวลาเรียนรู้การใช้แอป
15 โมเดลเก่าเจอข้อมูลใหม่แล้วแย่ = Maintainability
16 Explainability ให้คนเชื่อถือผล ไม่ใช่ลด latency/storage

ตัวเลข

160 bpm	threshold ที่ wearable ส่ง alert
5 s	กรอบเวลาส่ง alert สำเร็จ
12 ชม.	แบต wearable
640×480	กล้อง (= 307,200 px) · XRGB8888 = 4 ไบต์/px
1883	MQTT port
1 วิ	เขียน log (ทุก 1 วิ / เมื่อ status เปลี่ยน)
2,000 / 8,000 ฿	ต้นทุน farming / parking

สถานการณ์ → คำตอบ

สถานการณ์	ตอบ	สถานการณ์	ตอบ
อ่านป้าย→ไม้กั้นเปิดเอง	**Automation**	ผูกทะเบียน จ่าย+ปรับราคา	**Innovation**
นาฬิกาเตือนชีพจร > 120	**Notification**	กด "เปิดแอร์" ในแอป	**Control loop**
dashboard parking อัปเดตทันที	**Visualization**	Zigbee ฟาร์ม→Pi→cloud	**D2G**
Fitbit Cloud → Google Fit	**Data sharing (C2C)**	หูฟัง ↔ อุปกรณ์รอบข้าง	**D2D**
CCTV/รถอัตโนมัติ real-time	**Edge-only**	ฝึกโมเดลใหญ่ / big data	**Cloud-centric**
โรงงาน predictive maintenance	**Hybrid**	อุณหภูมิ+เวลาต่อเนื่อง	**Time-Series DB**
ทะเบียนรถ–เจ้าของ	**SQL**	เก็บรูป/วิดีโอจากกล้อง	**Object storage**
เซนเซอร์ไกลหลาย km ไฟน้อย	**LoRa**	อุปกรณ์สวมใส่ ใกล้ ไฟน้อยมาก	**BLE**

ตัวอย่างคำนวณ

- **fps** = 1/เวลาต่อเฟรม(s) → 0.05s=**20 fps** · 0.1s=10 · 25ms=40 · `FRAME_INTERVAL=1/TARGET_FPS`
- **พิกเซล** 640×480=**307,200** · ย่อครึ่ง 320×240=**76,800=1/4** · ทุก 3 เฟรม → detect 1/3 · รวมทั้งสอง ≈ **1/12**
- **Box filter 25×25 = 625** ช่อง ค่าละ 1/625 รวม=1 · ไม่หาร → สว่าง **625 เท่า** (ขาวโพลน) · k=15→225, k=49→2,401
- **Precision/Recall:** TP=90 FN=10 FP=20 → P=90/110=**0.82** · R=90/100=**0.90** · F1=2PR/(P+R)≈**0.86**
- **OCR:** `rotation_info=[0,90,-90]` → `[0]` = อ่านน้อยลง **~3 เท่า** · beam k ≈ k เท่าของ greedy · 3000×2000 → thumb 800 = **800×533**
- **เวลา:** 4a = (t1−t0)×**1000** ms · 4b−4a = เวลา classifier (เล็กมากเทียบ detect)
- **เกม:** 3 ท่า → คอมสุ่ม **1/3** ต่อท่า · 9 คู่ = **เสมอ 3 · ชนะ 3 · แพ้ 3**

พารามิเตอร์ EASYOCR

พารามิเตอร์	ความหมาย · tune
contrast_ths .05	contrast ต่ำกว่านี้ → อ่าน 2 รอบ เลือกที่มั่นใจกว่า
adjust_contrast .7	ระดับ contrast รอบสอง (1 = เดิม)
text_threshold .7	ความมั่นใจขั้นต่ำว่าเป็นข้อความ · ↑ ลดขยะ
low_text .3	เกณฑ์ข้อความจาง · ↓ จับตัวจางได้ (ขยะเพิ่ม)
link_threshold .4	เชื่อมอักษรเป็นกล่อง · สูง = แยก ต่ำ = รวม
allowlist	จำกัดอักขระ (ไทย+0–9) → ลดอ่านผิดประเภท
decoder	greedy / beamsearch → ใช้ greedy ให้เร็ว
rotation_info	ลองหมุนกล่อง → [0,90,-90] เปลี่ยนเป็น [0] เร็ว ~3 เท่า
paragraph / detail	False = ไม่รวมบรรทัด · 1 = คืน bbox+text+conf

ตัวที่เปลี่ยนผลชัดสุด: **contrast_ths · text_threshold · low_text** (+ `CONF_TH` กรองก่อนประกอบเลย)

คำถามอภิปรายในสไลด์

- **Lab1 protocol?** HTTP/TCP/IP บน Wi-Fi (browser↔Flask) · MJPEG (multipart) · SSH · CSI/MIPI — **ไม่มี MQTT**
- **Lab2 ประเมิน OCR?** exact-match ระดับป้าย + FAR/FRR + CER (ตัวช่วย) + confidence + latency + ความทนทาน + ต้นทุน
- **Lab2 เร่ง?** ย่อภาพ · greedy · rotation [0] · quantize · วัด ms ก่อน-หลัง
- **Lab3 ประเมิน RPS?** P/R/F1 รายท่า + confusion matrix · latency < 1s · robustness/fairness · ทรัพยากร · UX · ต้นทุน
- **Lab3 2 กล้องแบบ LINE?** ตรวจที่ edge ส่งแค่ป้ายท่า · sync เปิดท่าพร้อมกัน · ห้อง/ผู้ใช้ · latency/หลุด
- **smart parking?** Edge (ไม่ทั้นตอบเร็ว + ไม่พังเมื่อเน็ตล่ม) · **เบลอหน้า?** PDPA

Quick ref · โค้ด → ทำอะไร Lab 1–3

`CascadeClassifier(xml)` # โหลด Haar **ครั้งเดียวนอกลูป**
`detectMultiScale(gray,1.1,4)` # → `[(x,y,w,h)]` · ว่าง = ไม่เจอ
`np.ones((25,25))/(25*25)` # box kernel · ผลรวม = 1 (ไม่หาร → ขาวโพลน)
`filter2D(roi,-1,k)` # โก kernel (correlation) · -1 = depth เดิม
`cvtColor(BGR2GRAY)` / `BGR2RGB` # เทาก่อน detect / แก้สีเพี้ยนให้ MediaPipe
`flip(frame,1)` # กระจกซ้าย-ขวา
`rectangle(…)` · `putText(…,LINE_AA)` · `circle(…,-1)` # กรอบ / ข้อความ / จุด (-1=ทึบ)
`resize(img,None,fx,fy)` # ย่อภาพ (งาน conv ตามจำนวนพิกเซล)
`imencode(".jpg",f,[IMWRITE_JPEG_QUALITY,80])` # → JPEG bytes
`yield b"--frame\r\n…" + Response(gen(),"multipart/x-mixed-replace")` # สตรีม MJPEG ขึ้นเว็บ
`perf_counter()` # 4a = detect · 4b = +classifier · 4b−4a = classifier
`isoformat(timespec="seconds")` # ISO-8601 + timezone (log)
`Reader(['th'],gpu=False)` # โหลดโมเดล OCR ครั้งเดียว · `readtext()` → `[(bbox,text,conf)]`
`Contrast(1.25)` · `UnsharpMask(1.0,120,3)` # contrast / sharpen ก่อน OCR
`Image.open(f).convert("RGB")` · `thumbnail((800,800))` # อ่านเป็น PIL · ย่อคงสัดส่วน
`ImageDraw.Draw` · `ImageFont.truetype(p,22)` # วาดข้อความ · โหลดฟอนต์
`sqlite3 (?,…)` · `commit()` # กัน injection · ลืม commit = ไม่บันทึก
`mp.Image(SRGB,data=rgb)` + `detect()` · `lm.x*W` # หามือ 21 จุด (0–1) → พิกเซล
`@app.route(…,methods=["POST"])` # GET → 405
`random.choice([...])` · `render_template("game.html",…)` # สุ่มท่าคอม · ส่งค่าเข้า HTML
Python: `f"{x:.2f}"` · `[r for r in rs if r[2]>=th]` · `lambda b:b[0]` · `.sort(key=…)` · `x % N == 0` · `with open(p,"a") as f` · `if __name__=="__main__"` · `h,w,_ = frame.shape`
Flask: `Flask(__name__)` · `@app.route(p,methods=["GET","POST"])` · `request.files["image"]` · `abort(400,"No file")` · `render_template / render_template_string` · `app.run(host="0.0.0.0",port=5000)`